

Assignment Number - 1

AIM: Introduction to Database Creation Using MySQL/PostgreSQL

Title:

Objective: Set up and create a database using MySQL/PostgreSQL.

Task: Entities and Relationships:

- Student (StudentID, Name, Email, Age, Address)
- Course (CourseID, CourseName, Credits, InstructorID)
- Instructor (InstructorID, Name, Email, Department)
- Enrollment (EnrollmentID, StudentID, CourseID, EnrollmentDate)

Relationships:

- A student can enroll in multiple courses.
- A course can have multiple students.
- An instructor teaches one or more courses.

Objective: To understand the fundamentals of database creation by designing and implementing a relational database using MySQL or PostgreSQL, based on given entities and their relationships.

Outcome: After completing this lab, the student will be able to:

- Understand the concept of databases and relational models
- Identify entities, attributes, and relationships
- Create tables using SQL
- Define primary keys and foreign keys
- Implement relationships between tables
- Insert and retrieve data from a database

Software Required:

MySQL Server / PostgreSQL Server

MySQL Workbench / pgAdmin

Operating System: Windows / Linux / macOS

Theory: Introduction to MySQL

MySQL is an open-source Relational Database Management System (RDBMS) used to store, organize, and manage large amounts of structured data efficiently. It uses

Structured Query Language (SQL) as the standard language for performing database operations such as creating databases, inserting data, updating records, and retrieving information.

In MySQL, data is stored in the form of tables, which consist of rows (records) and columns (attributes). Each table represents an entity, and relationships between tables are established using primary keys and foreign keys. This relational structure helps maintain data accuracy and reduces redundancy.

MySQL follows a client-server architecture. The MySQL server manages the databases and processes SQL queries, while clients such as MySQL Workbench or command-line tools send queries to the server and display results. This architecture allows multiple users to access and work with the database simultaneously.

One of the key strengths of MySQL is its performance and scalability. It can handle large databases and high traffic efficiently, making it suitable for web-based and enterprise applications. MySQL also provides security features such as user authentication, access control, and data encryption to protect sensitive information.

MySQL supports multiple storage engines like InnoDB and MyISAM. InnoDB is commonly used because it supports transactions, foreign keys, and data integrity, which are essential for reliable database operations.

Due to its simplicity, reliability, and wide community support, MySQL is extensively used in applications such as student management systems, e-commerce platforms, banking systems, and content management systems.

2. Installation of MySQL

MySQL can be installed on various operating systems, including Windows, macOS, and Linux. Here are the general steps to install MySQL:

Windows:

1. Download the MySQL installer from the official website.
2. <https://dev.mysql.com/downloads/installer/>
3. Run the installer and follow the on-screen instructions.
4. Choose the installation type (Typical, Complete, or Custom). Recommended Custom.
5. Set a root password for the MySQL server.

3. MySQL Installation in Ubuntu

MySQL can be easily installed on **Ubuntu Linux** using the **APT (Advanced Package Tool)** package manager. Ubuntu provides MySQL packages in its official repositories, which makes the installation process simple, secure, and reliable. After installation, basic configuration is performed to secure the database server and verify its proper functioning.

Steps to Install MySQL on Ubuntu

1. Update the Package Index

This ensures that the system has the latest information about available packages.

```
sudo apt update
```

2. Install MySQL Server

Install the MySQL server package using the following command:

```
sudo apt install mysql-server
```

During installation, required dependencies will be installed automatically.

3. Check MySQL Service Status

After installation, verify that the MySQL service is running:

```
sudo systemctl status mysql
```

4. Secure MySQL Installation

Run the security script to improve MySQL security:

```
sudo mysql_secure_installation
```

This script helps to:

- Set a root password

- Remove anonymous users

- Disable remote root login

- Remove test databases

5. Login to MySQL

To access the MySQL shell:

```
sudo mysql
```

6. Verify Installation

Inside the MySQL prompt, check the version:

```
SELECT VERSION();
```

If the version is displayed, MySQL is installed successfully.

4. Database

A database is an organized collection of data that allows easy access, management, and updating of information. Relational databases store data in tables consisting of rows and columns.

5. Entities and Attributes

An **entity** represents a real-world object. Each entity has attributes that describe it.

Student

StudentID (Primary Key)

Name

Email

Age

Address

Instructor

InstructorID (Primary Key)

Name

Email

Department

Course

CourseID (Primary Key)

CourseName

Credits

InstructorID (Foreign Key)

Enrollment

EnrollmentID (Primary Key)
StudentID (Foreign Key)
CourseID (Foreign Key)
EnrollmentDate

Relationships

A student can enroll in multiple courses (One-to-Many)
A course can have multiple students (One-to-Many)
An instructor teaches one or more courses (One-to-Many)

The Enrollment table is used to manage the many-to-many relationship between Student and Course.

SQL Table Creation

```
CREATE TABLE Student (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Email VARCHAR(50),  
    Age INT,  
    Address VARCHAR(100)  
);
```

```
CREATE TABLE Instructor (  
    InstructorID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Email VARCHAR(50),  
    Department VARCHAR(50)  
);
```

```
CREATE TABLE Course (  
    CourseID INT PRIMARY KEY,  
    CourseName VARCHAR(50),  
    Credits INT,  
    InstructorID INT,  
    FOREIGN KEY (InstructorID) REFERENCES Instructor(InstructorID)  
);
```

```
CREATE TABLE Enrollment (  
    EnrollmentID INT PRIMARY KEY,  
    StudentID INT,  
    CourseID INT,  
    EnrollmentDate DATE  
);
```

```
EnrollmentID INT PRIMARY KEY,  
StudentID INT,  
CourseID INT,  
EnrollmentDate DATE,  
FOREIGN KEY (StudentID) REFERENCES Student(StudentID),  
FOREIGN KEY (CourseID) REFERENCES Course(CourseID)  
);
```

Conclusion

In this lab, we successfully designed and created a relational database using MySQL/PostgreSQL. We learned how to identify entities, define attributes, establish relationships, and implement them using SQL commands. This experiment helps in understanding real-world database applications such as student management systems.

Oral Questions

1. What is a database?
2. What is a primary key?
3. What is a foreign key?
4. What is the purpose of the Enrollment table?
5. Explain one-to-many and many-to-many relationships.
6. What is SQL?
7. Difference between MySQL and PostgreSQL?
8. Why are relationships important in databases?

Assignment Number - 2

AIM: Views and Indexing – Optimizing a Student Course Management System for Faster Query Performance.

Problem Statement: Views and Indexing

Optimizing a Student Course Management System with Views and Indexing for Faster Query Performance.

Objective: Understand views and indexing for performance optimization.

Task:

- Create views for easy reporting on students and course details.
- Update and delete records using views where applicable.
- Use indexing to speed up searches and measure query performance improvements.

Objective: To understand the concept of Views and Indexing in a database and apply them to optimize query performance in a Student Course Management System.

Outcome

Views were created to simplify student and course reporting.
Records were updated and deleted using updatable views.
Indexes improved search speed and query performance.
Overall database efficiency was enhanced using views and indexing.

Software Requirements

Database: MySQL / PostgreSQL / Oracle

SQL Client: MySQL Workbench / pgAdmin / SQL*Plus

Theory:

Views:

A View is a virtual table created using a SQL query. It does not store data physically but displays data from one or more tables. A VIEW is a virtual table based on the result-set of an SQL query. It contains rows and columns, just like a real table, but it doesn't store data itself. The data in a view is derived from the base tables at the time the view is accessed.

Why use Views?

Simplicity: Simplify complex queries (e.g., hiding multi-table joins) into a single, easy-to-query virtual table.

Security: Grant users access to specific data through a view, without giving them direct access to the underlying tables. You can expose only certain columns or rows.

Data Abstraction: Present data in a way that is logical to the user, regardless of how the data is physically stored.

Consistency: Provide a consistent view of data, even if the underlying schema changes (as long as the view's definition is updated).

Basic Syntax:

```
CREATE VIEW ViewName AS
```

```
SELECT column1, column2, ...
```

```
FROM table_name
```

```
WHERE condition;
```

Example: Create a view to see only the FirstName, LastName, and Major of students, hiding their age or other sensitive info.


```
CREATE VIEW Student_Majors_View AS

SELECT FirstName, LastName, Major

FROM Students

WHERE Major IS NOT NULL; -- Only show students with a known major
```

Now, you can query this view just like a table:

```
SELECT * FROM Student_Majors_View WHERE Major = 'Computer Science';
```

Output (Example):

FirstName	LastName	Major
Alice	Smith	Computer Science
Grace	Green	Computer Science

Advantages of Views:

- Simplifies complex queries
- Improves data security
- Useful for reporting
- Allows controlled updates and deletions

Indexing

An Index is a database object that improves the speed of data retrieval operations. An INDEX is a special lookup table that the database search engine can use to speed up data retrieval. Think of it like the index at the back of a book: instead of reading the whole book to find a topic, you go to the index, find the page number, and go directly there.

Why use Indexes?

- Faster SELECT queries: Dramatically speeds up queries, especially with WHERE clauses, JOIN conditions, and ORDER BY clauses on indexed columns.
- Unique enforcement: UNIQUE and PRIMARY KEY Constraints automatically create indexes to ensure uniqueness.

Basic Syntax:

```
CREATE INDEX IndexName  
  
ON table_name (column1, column2, ...);
```

Example: Create an index on the LastName column of the Students table to speed up searches by last name.

```
CREATE INDEX IX_Students_LastName  
  
ON Students (LastName);
```

You can also create a UNIQUE INDEX to ensure all values in the indexed column(s) are unique:

```
CREATE UNIQUE INDEX UQ_Students_Email  
  
ON Students (Email); -- If you had an Email column
```

When to Index?

- Columns frequently used in WHERE clauses.
- Columns frequently used in JOIN conditions.
- Columns frequently used in ORDER BY clauses.
- Columns used in aggregate functions where GROUP BY is applied.
- Foreign key columns.

Advantages of Indexing:

- Faster search operations
- Efficient filtering and sorting
- Reduced query execution time

Drawbacks:

- Slower INSERT, UPDATE, DELETE: Maintaining indexes adds overhead to data modification operations because the index itself must be updated.
- Disk Space: Indexes consume disk space

Step to solve the given problem

Create Database and Use It

Table creation

Sample data insertion

Views creation

Indexing creation

Example queries with performance check

- Sample Examples:

- Query 1: Get all courses a student is enrolled in

- Query 2: Get courses with more than 1 student

- Update and Delete Examples

- Check updated views

Conclusion

The experiment demonstrated that:

Views simplify complex queries and improve data accessibility.

Updatable views allow safe modification of records.

Indexing enhances query performance and reduces execution time.

Thus, Views and Indexing play a crucial role in database optimization.

Oral Questions:

1. What is a View?
2. Can you update data through a view?
3. What is an Index?
4. Benefits of Indexing?
5. Difference between a Table and a View?

Assignment Number - 3

AIM: Advanced SQL Querying in a Student Course Management System Using Filters, Sorting, Aggregation, and Nested Subqueries

Problem Statement:

SQL Queries with Filters and Sorting

Advanced Querying in a Student Course Management System Using Filters, Sorting, and Nested Subqueries

Objective: Use SQL filters and sorting clauses.

Task:

- Filtering Data with WHERE (Find students above a certain age, filter by instructor)
- Sorting with ORDER BY (Alphabetically, by credits)
- Aggregation with GROUP BY (Student count per course, average age)
- Nested Subqueries for Complex Queries

Objective:

To practice retrieving data from multiple tables using SQL.

To learn how to filter data using WHERE clauses.

To sort query results with ORDER BY.

To perform aggregation using GROUP BY and functions like COUNT() and AVG().

To write nested subqueries for complex data retrieval.

Outcome:

After completing this lab, students will be able to:

Filter data based on conditions.

Sort query results in ascending or descending order.

Aggregate data to get counts, averages, or summaries.

Write nested queries to solve complex questions.

Understand relational database concepts and SQL query optimization.

Software Requirements

Database Management System (DBMS): MySQL / MariaDB / PostgreSQL / Oracle / SQL Server

SQL Client / IDE (Optional but Recommended): MySQL Workbench, pgAdmin, DBeaver, SQL Server Management Studio (SSMS)

Operating System: Windows 10 or later, macOS, or Linux

Hardware Requirements (Minimal):

Processor: Intel i3 or equivalent

RAM: 4 GB or more

Storage: 2 GB free space for DBMS installation

Display: 1024×768 resolution or higher

Theory:

1. Introduction to SQL

Structured Query Language (SQL) is the standard language used to communicate with relational databases. It allows users to store, retrieve, update, and delete data efficiently.

A relational database organizes data into tables (also called relations), where each table consists of rows (records) and columns (fields). SQL is essential for managing this data and performing operations like filtering, sorting, aggregating, and combining data from multiple tables.

2. Basic SQL Commands

SQL commands are generally divided into the following categories:

1. **Data Definition Language (DDL):** Used to define and modify database structure

Examples: CREATE TABLE, ALTER TABLE, DROP TABLE

2. **Data Manipulation Language (DML):** Used to manipulate the data stored in tables

Examples: SELECT, INSERT, UPDATE, DELETE

3. **Data Control Language (DCL):** Used to control access to data

Examples: GRANT, REVOKE

4. **Transaction Control Language (TCL):** Used to manage transactions

Examples: COMMIT, ROLLBACK

3. filtering and sorting

Database filtering and sorting are two distinct operations used to manipulate and retrieve data from a database, but they serve different purposes.

Filtering is the process of selecting a subset of records from a database based on specific conditions or criteria. It reduces the number of records returned by querying only those that match the given rules. Filtering is typically done using clauses like WHERE in SQL.

Example: Suppose you have a database table of employees with columns like Name, Department, and Salary. If you want to retrieve only employees from the "Sales" department, you would use a filter:

```
SELECT * FROM Employees WHERE Department = 'Sales';
```

This query returns only the rows where the Department is "Sales," excluding all other departments.

Sorting, on the other hand, arranges the retrieved records in a specific order (ascending or descending) based on one or more columns. It does not reduce the number of records but

organizes them for better readability or analysis. Sorting is done using clauses like ORDER BY in SQL.

Example: If you want to retrieve all employees but display them in order of their salary (from highest to lowest), you would use:

```
SELECT * FROM Employees ORDER BY Salary DESC;
```

This query returns all employees, but the results are sorted by the Salary column in descending order.

Key Differences:

1. **Purpose:** Filtering selects specific records based on conditions, while sorting arranges records in a particular order.
2. **Result Impact:** Filtering reduces the number of records, whereas sorting does not change the count but reorganizes them.
3. **SQL Clauses:** Filtering uses WHERE, while sorting uses ORDER BY.

In cloud-based database services, such as those provided by Tencent Cloud, these operations are fundamental for efficient data retrieval. Tencent Cloud's **TencentDB** (e.g., MySQL, PostgreSQL, or MongoDB-compatible databases) supports powerful filtering and sorting capabilities, allowing users to optimize queries for performance and scalability. For large datasets, indexing and optimized queries can further enhance efficiency when applying filters and sorting.

4.Order By and Group By Clause:

The ORDER BY clause is used to **sort the rows returned by a query** in **ascending (ASC)** or **descending (DESC)** order based on one or more columns.

Default: ASC (ascending) if you don't specify.

Can sort by **one or multiple columns**.

Syntax:

```
SELECT column1, column2, ...
```

```
FROM table_name
```

```
ORDER BY column1 ASC|DESC, column2 ASC|DESC;
```

Example 1: List all students alphabetically

```
SELECT name, age FROM Students ORDER BY name ASC;
```

Example 2: List courses by credits from highest to lowest

```
SELECT course_name, credits FROM Courses ORDER BY credits DESC;
```

Key Points:

- ORDER BY is applied **after all filtering (WHERE)** and joins.
- Useful for **reports, leaderboards, or sorted lists**.

GROUP BY:

The GROUP BY clause is used to **group rows that have the same value in one or more columns** and often works with **aggregate functions** like COUNT(), AVG(), SUM(), MAX(), MIN().

Syntax:

```
SELECT column1, AGG_FUNC(column2)
```

```
FROM table_name
```

```
GROUP BY column1;
```

Example 1: Count the number of students in each major

```
SELECT major, COUNT(student_id) AS student_count FROM Students GROUP BY major;
```

Key Points:

GROUP BY reduces multiple rows into summary rows.

Can be combined with HAVING to filter groups:

```
SELECT major, COUNT(student_id) AS student_count
```


FROM Students

GROUP BY major

HAVING COUNT(student_id) > 1; -- only majors with more than 1 student

How to Solve Questions Step by Step:

1. Understand the Question – Determine whether it requires filtering, sorting, aggregation, or subqueries.
2. Identify Tables – Locate the relevant tables containing required information.
3. Choose SQL Clauses – Use WHERE, ORDER BY, GROUP BY, or subqueries as needed.
4. Write the Query Stepwise
 - Start simple with SELECT
 - Apply filters with WHERE
 - Sort using ORDER BY
 - Aggregate using GROUP BY
 - Use nested queries if results depend on another query
5. Test and Verify – Run the query and check the results for correctness.

Conclusion:

SQL allows for efficient data retrieval using filters, sorting, aggregation, and nested subqueries.

Stepwise query building ensures accuracy and clarity.

Mastering these techniques is essential for managing relational databases and generating meaningful reports.

Oral Questions:

1. What is the difference between WHERE and HAVING clauses?
2. How does ORDER BY affect performance?
3. Explain GROUP BY with an example.
4. What is a nested subquery and why use it?

5. Can a subquery return multiple values? How do you handle it?
6. How can grades (A, B, C) be converted to numeric values in SQL?
7. Difference between INNER JOIN and LEFT JOIN.

Assignment Number - 4

AIM : Multi-Table Joins in a Customer Order Management System Using SQL INNER, LEFT, RIGHT, and FULL OUTER JOINS .

Problem Statement: Advanced SQL Querying Using Multi-Table Joins. Develop a Customer Order Management System and retrieve meaningful data by combining information from multiple tables using SQL joins.

Objective: Use SQL join operations to retrieve and analyze data from related tables.

Task:

- **INNER JOIN:** Retrieve all customer orders (only customers who have placed orders).
- **LEFT JOIN:** Retrieve all customers, including those who haven't placed any orders.
- **RIGHT JOIN:** Retrieve all orders, including those placed by customers not present in the Customers table.
- **FULL OUTER JOIN:** Retrieve a complete list of customers and orders, including unmatched records from both tables.

Objectives of the Experiment:

To practice retrieving data from multiple tables using SQL JOIN operations.

To understand relationships between tables using primary and foreign keys.

To learn and apply INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN.

To analyze real-world scenarios using relational database concepts.

To improve query-writing and database design skills.

Outcome:

After completing this lab, students will be able to:

Combine data from multiple tables using JOIN operations.

Identify differences between INNER, LEFT, RIGHT, and FULL OUTER JOIN.

Retrieve complete and partial datasets based on join types.

Apply joins in real-world database applications like order management systems.

Understand relational integrity and query optimization concepts.

Software Requirements:

Database Management System (DBMS): MySQL / MariaDB / PostgreSQL / Oracle / SQL Server

SQL Client / IDE (Optional but Recommended):

MySQL Workbench, pgAdmin, DBeaver, SQL Server Management Studio (SSMS)

Operating System: Windows 10 or later, macOS, or Linux

Hardware Requirements (Minimal):

Processor: Intel i3 or equivalent

RAM: 4 GB or more

Storage: 2 GB free space for DBMS installation

Display: 1024 × 768 resolution or higher

Theory:

1. Introduction to SQL Joins

In relational databases, data is stored across multiple tables to reduce redundancy and maintain data integrity. SQL JOIN operations are used to combine rows from two or more tables based on a related column, usually a primary key–foreign key relationship.

In a Customer Order Management System, customer details are stored in one table, while order details are stored in another. Joins help retrieve meaningful information such as which customer placed which order.

2. Types of SQL Joins

SQL Joins are used to combine data from two or more tables based on a related column, usually a primary key-foreign key relationship. Different types of joins return different result sets depending on how matching and non-matching records are handled.

1. INNER JOIN

An INNER JOIN returns only those records where there is a matching value in both tables involved in the join. Records that do not have a match in either table are excluded from the result set. This type of join is commonly used when we want only related data from both tables.

Example Use Case: Retrieve customers who have placed at least one order.

Key Characteristics:

- Displays only matching records from both tables
- Excludes all unmatched rows
- Most commonly used join in SQL

2. LEFT JOIN (LEFT OUTER JOIN)

A LEFT JOIN returns all records from the left table and the matching records from the right table. If there is no matching record in the right table, the result will contain NULL values for the right table's columns. This join is useful when it is important to include all records from the main table, regardless of whether related data exists.

Example Use Case: Retrieve all customers, including those who have not placed any orders.

Key Characteristics:

- All records from the left table are included

Matching records from the right table are shown

Non-matching rows from the right table appear as NULL

3. RIGHT JOIN (RIGHT OUTER JOIN)

A RIGHT JOIN returns all records from the right table and the matching records from the left table. If there is no matching record in the left table, NULL values are displayed for the left table's columns. This join ensures that all records from the secondary table are included in the result.

Example Use Case: Retrieve all orders, including those placed by customers whose details are missing from the Customers table.

Key Characteristics:

- All records from the right table are included

- Matching records from the left table are displayed

- Unmatched records from the left table contain NULL values

4. FULL OUTER JOIN

A FULL OUTER JOIN returns all records from both tables, whether or not there is a matching record in the other table. When no match exists, NULL values are shown for the missing side. This join provides a complete view of data from both tables.

Example Use Case: Retrieve a complete list of customers and orders, including customers without orders and orders without customers.

Key Characteristics:

- Includes all records from both tables

- Displays matching rows where available

- Unmatched rows from either table contain NULL values

- Not directly supported in MySQL (can be simulated using UNION)

Procedure:

Step 1: Create Tables

```
CREATE TABLE Customers (  
    customer_id INT PRIMARY KEY,  
    customer_name VARCHAR(100),  
    email VARCHAR(100),  
    city VARCHAR(50)  
);
```

```
CREATE TABLE Orders (  
    order_id INT PRIMARY KEY,  
    order_date DATE,  
    amount DECIMAL(10,2),  
    customer_id INT  
);
```

Step 2: Insert Data

```
INSERT INTO Customers VALUES  
(1, 'Amit Sharma', 'amit@gmail.com', 'Pune'),  
(2, 'Neha Patil', 'neha@gmail.com', 'Mumbai'),  
(3, 'Rahul Mehta', 'rahul@gmail.com', 'Delhi'),  
(4, 'Sneha Kulkarni', 'sneha@gmail.com', 'Pune'),  
(5, 'Karan Desai', 'karan@gmail.com', 'Ahmedabad');
```

```
INSERT INTO Orders VALUES  
(101, '2024-01-10', 2500.00, 1),  
(102, '2024-01-12', 1800.50, 2),  
(103, '2024-01-15', 3200.00, 1),  
(104, '2024-01-18', 1500.00, 6),  
(105, '2024-01-20', 2750.75, 3);
```

Step 3: SQL Queries

1. INNER JOIN - Retrieve customers who have placed orders.

```
SELECT Customers.customer_id, customer_name, order_id, order_date
```

```
FROM Customers
INNER JOIN Orders
ON Customers.customer_id = Orders.customer_id;
```

2. LEFT JOIN

Retrieve all customers including those who have not placed orders.

```
SELECT Customers.customer_id, customer_name, order_id
FROM Customers
LEFT JOIN Orders
ON Customers.customer_id = Orders.customer_id;
```

3. RIGHT JOIN

Retrieve all orders including those without matching customer records.

```
SELECT customer_name, order_id, order_date
FROM Customers
RIGHT JOIN Orders
ON Customers.customer_id = Orders.customer_id;
```

4. FULL OUTER JOIN (Using UNION)

Retrieve all customers and all orders.

```
SELECT Customers.customer_id, customer_name, order_id
FROM Customers
LEFT JOIN Orders
ON Customers.customer_id = Orders.customer_id
UNION
SELECT Customers.customer_id, customer_name, order_id
FROM Customers
RIGHT JOIN Orders
ON Customers.customer_id = Orders.customer_id;
```

Result:

All SQL join queries were executed successfully.

The output displayed correct records based on the join type, including matched and unmatched records from both tables.

Conclusion:

SQL JOIN operations are essential for retrieving related data from multiple tables.

Different join types serve different real-world use cases.

Understanding joins improves database efficiency and reporting accuracy.

Joins form the backbone of real-world applications like banking, e-commerce, and ERP systems.

Oral Questions:

1. What is the purpose of SQL JOINS?
2. Difference between INNER JOIN and LEFT JOIN?
3. When should RIGHT JOIN be used?
4. Why does MySQL not support FULL OUTER JOIN directly?
5. How can FULL OUTER JOIN be simulated in MySQL?
6. What is a foreign key and why is it important in joins?
7. How do joins affect query performance?

Assignment Number - 5

AIM: PL/SQL Procedures and Functions in a Sales Management System for Efficient Data Handling .

Problem Statement: Design and Implement a Sales Management System Using PL/SQL Stored Procedures and Functions. Develop a Sales Management System and perform database operations using stored procedures and functions to efficiently insert data and compute total sales.

Objective:

To implement PL/SQL stored procedures and functions for managing sales data and performing calculations within the database.

Task:

Create a stored procedure to add sales records to a table.

Create a function to calculate and return the total sales amount.

Objectives of the Experiment:

- To understand the concept of PL/SQL procedures and functions.
- To implement stored procedures for inserting data into tables.
- To implement functions for performing calculations and returning values.
- To reduce code redundancy and improve database performance.
- To apply PL/SQL in real-world database applications.

Outcome:**After completing this lab, students will be able to:**

- Create and execute PL/SQL stored procedures.
- Create and use functions to return computed values.
- Understand the difference between procedures and functions.
- Apply PL/SQL concepts in real-time database systems.
- Improve efficiency and security of database operations.

Software Requirements:

- Database Management System (DBMS): Oracle Database / PostgreSQL (PL/pgSQL)
- SQL Client / IDE: SQL*Plus, Oracle SQL Developer, pgAdmin
- Operating System: Windows 10 or later, macOS, or Linux

Hardware Requirements (Minimal):

- Processor: Intel i3 or equivalent
- RAM: 4 GB or more
- Storage: 2 GB free space
- Display: 1024 × 768 resolution or higher

Theory:**1. Introduction to PL/SQL**

PL/SQL (Procedural Language / SQL) is Oracle's procedural extension of SQL. It allows developers to write blocks of code that include procedural constructs such as loops, conditions, procedures, and functions along with SQL statements.

PL/SQL improves:

- Performance (by reducing network traffic)
- Security
- Code reusability
- Maintainability

2. Stored Procedures

A stored procedure is a named PL/SQL block that performs a specific task. It can accept input parameters and execute SQL statements but does not return a value directly.

Advantages:

- Improves performance
- Enhances modular programming
- Reduces code duplication

3. Functions

A function is a named PL/SQL block that returns a single value. Functions are mainly used for computations and can be called within SQL statements.

Advantages:

- Returns computed results
- Can be reused in SQL queries
- Improves clarity and efficiency

Procedure:

Step 1: Create Sales Table

```
CREATE TABLE Sales (  
    sale_id INT PRIMARY KEY,  
    product_name VARCHAR(100),  
    quantity INT,
```

```
price DECIMAL(10,2),  
sale_date DATE  
);
```

Step 2: Create Stored Procedure

Stored Procedure to Insert Sales Data

```
CREATE OR REPLACE PROCEDURE add_sale (  
    p_sale_id IN INT,  
    p_product_name IN VARCHAR,  
    p_quantity IN INT,  
    p_price IN DECIMAL,  
    p_sale_date IN DATE  
) AS  
BEGIN  
    INSERT INTO Sales  
    VALUES (p_sale_id, p_product_name, p_quantity, p_price, p_sale_date);  
  
    COMMIT;  
END;  
/
```

Step 3: Execute Stored Procedure

```
EXEC add_sale(1, 'Laptop', 2, 55000, DATE '2024-01-10');  
EXEC add_sale(2, 'Mouse', 5, 500, DATE '2024-01-12');  
EXEC add_sale(3, 'Keyboard', 3, 1200, DATE '2024-01-15');
```

Step 4: Create Function

Function to Calculate Total Sales Amount

```
CREATE OR REPLACE FUNCTION get_total_sales
RETURN DECIMAL AS
    total_sales DECIMAL(10,2);
BEGIN
    SELECT SUM(quantity * price)
    INTO total_sales
    FROM Sales;

    RETURN total_sales;
END;
/
```

Step 5: Execute Function

```
SELECT get_total_sales FROM dual;
```

Result:

The stored procedure successfully inserted sales records into the Sales table.
The function correctly calculated and returned the total sales amount.

Conclusion:

PL/SQL procedures help in performing database operations efficiently.
Functions are useful for returning computed values.
Using PL/SQL improves performance, security, and maintainability.
Stored procedures and functions are widely used in enterprise applications.

Oral Questions:

1. What is PL/SQL?
2. Difference between procedure and function?
3. Can a function be used inside a SELECT statement?
4. Why is COMMIT used in procedures?

5. What happens if a function does not return a value?
6. Can procedures accept parameters?
7. Where are PL/SQL procedures used in real-world applications?

Assignment Number - 6

AIM: Triggers and Cursors in an Employee Management System for Audit Logging and Efficient Data Processing Using PL/SQL.

Problem Statement:

Develop an Employee Management System Using PL/SQL Triggers and Cursors.

Design and implement triggers to maintain audit logs for employee data changes and use cursors to process employee records for conditional salary updates.

Objective:

To implement PL/SQL triggers and cursors for monitoring data changes and performing row-by-row data processing efficiently.

Task:

Create a trigger on the Employees table to log INSERT, UPDATE, and DELETE operations into an Employee_Audit table.

Use a cursor to iterate through the Employees table and apply a 10% salary increment for employees working in the IT department.

Objectives of the Experiment:

To understand the working of PL/SQL triggers.

To implement audit logging using database triggers.

To understand cursors for row-by-row data processing.

To apply conditional logic using cursors.

To use PL/SQL for real-world database automation tasks.

Outcome:

After completing this lab, students will be able to:

Create and use triggers for automatic data auditing.

Track INSERT, UPDATE, and DELETE operations in database tables.

Use cursors to process records sequentially.

Apply conditional updates using PL/SQL logic.

Implement enterprise-level database automation techniques.

Software Requirements:

Database Management System (DBMS): Oracle Database

SQL Client / IDE: SQL*Plus / Oracle SQL Developer

Operating System: Windows 10 or later, macOS, or Linux

Hardware Requirements (Minimal):

Processor: Intel i3 or equivalent

RAM: 4 GB or more

Storage: 2 GB free space

Display: 1024 × 768 resolution or higher

Theory :

1. Introduction to Triggers

A trigger is a special PL/SQL block that is automatically executed when a specified event (INSERT, UPDATE, or DELETE) occurs on a table or view. Triggers are commonly used for audit logging, enforcing business rules, and maintaining data integrity.

2. Introduction to Cursors

A cursor is a PL/SQL construct used to process query results row by row. Cursors are useful when complex logic needs to be applied individually to each record returned by a query.

3. Types of Triggers

BEFORE Trigger

AFTER Trigger

ROW-level Trigger

STATEMENT-level Trigger

In this experiment, an AFTER ROW trigger is used for audit logging.

4. Types of Cursors

Implicit Cursor

Explicit Cursor

In this experiment, an explicit cursor is used for salary processing.

Procedure:

Step 1: Create Employees Table

```
CREATE TABLE Employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(100),  
    department VARCHAR(50),
```



```
    salary DECIMAL(10,2)
);
```

Step 2: Create Employee_Audit Table

```
CREATE TABLE Employee_Audit (
    audit_id INT GENERATED ALWAYS AS IDENTITY,
    emp_id INT,
    operation VARCHAR(10),
    old_salary DECIMAL(10,2),
    new_salary DECIMAL(10,2),
    action_date DATE
);
```

Step 3: Insert Sample Data

```
INSERT INTO Employees VALUES
(1, 'Rahul Patil', 'IT', 50000),
(2, 'Neha Sharma', 'HR', 45000),
(3, 'Amit Verma', 'IT', 60000),
(4, 'Sneha Kulkarni', 'Finance', 55000);
```

Step 4: Create Trigger for Audit Logging

```
CREATE OR REPLACE TRIGGER emp_audit_trigger
AFTER INSERT OR UPDATE OR DELETE ON Employees
FOR EACH ROW
BEGIN
    IF INSERTING THEN
        INSERT INTO Employee_Audit
        (emp_id, operation, old_salary, new_salary, action_date)
        VALUES (:NEW.emp_id, 'INSERT', NULL, :NEW.salary, SYSDATE);

    ELSIF UPDATING THEN
```

```

INSERT INTO Employee_Audit
(emp_id, operation, old_salary, new_salary, action_date)
VALUES (:NEW.emp_id, 'UPDATE', :OLD.salary, :NEW.salary, SYSDATE);

ELSIF DELETING THEN
    INSERT INTO Employee_Audit
    (emp_id, operation, old_salary, new_salary, action_date)
    VALUES (:OLD.emp_id, 'DELETE', :OLD.salary, NULL, SYSDATE);
END IF;
END;
/

```

Step 5: Create Cursor for Salary Increment

```

DECLARE
    CURSOR emp_cursor IS
        SELECT emp_id, salary
        FROM Employees
        WHERE department = 'IT';

    v_emp_id Employees.emp_id%TYPE;
    v_salary Employees.salary%TYPE;
BEGIN
    OPEN emp_cursor;

    LOOP
        FETCH emp_cursor INTO v_emp_id, v_salary;
        EXIT WHEN emp_cursor%NOTFOUND;

        UPDATE Employees
        SET salary = v_salary * 1.10
        WHERE emp_id = v_emp_id;
    
```

```
END LOOP;

CLOSE emp_cursor;
COMMIT;
END;
/
```

Result:

The trigger successfully logged all INSERT, UPDATE, and DELETE operations into the Employee_Audit table.

The cursor correctly processed IT department employees and updated their salaries by 10%.

Conclusion:

Triggers automate audit logging without user intervention.

Cursors allow row-by-row data processing in PL/SQL.

Combining triggers and cursors improves data integrity and automation.

These concepts are widely used in enterprise employee management systems.

Oral Questions:

1. What is a trigger in PL/SQL?
2. Difference between BEFORE and AFTER trigger?
3. What is a cursor?
4. Why are cursors required in PL/SQL?
5. What is the use of :OLD and :NEW keywords?
6. Can triggers be rolled back?
7. Difference between implicit and explicit cursors?

Assignment Number - 7

Title: NoSQL Database Introduction – Implementation of MongoDB CRUD Operations, Aggregation, and Indexing.

Aim: To study and implement MongoDB, a NoSQL database, and perform CRUD operations, aggregation queries for data analysis, and indexing techniques to improve query performance.

Problem Statement: Design and implement a MongoDB-based NoSQL system to store user-related data. Perform CRUD operations, analyze data using aggregation pipelines, and enhance query performance using indexing.

Objectives:

- To understand the fundamentals of NoSQL databases.
- To create and manage MongoDB databases and collections.
- To perform Create, Read, Update, and Delete operations.
- To analyze data using MongoDB aggregation pipelines.
- To implement indexing and evaluate its effect on performance.

Outcomes:

After completing this experiment, students will be able to:

- Explain the difference between SQL and NoSQL databases
- Design document-based data models in MongoDB
- Perform CRUD operations using MongoDB shell
- Write aggregation queries for analytical processing
- Apply indexing techniques for query optimization

Software Requirements:

- MongoDB Community Server
- MongoDB Compass / Mongo Shell (mongosh)
- Operating System: Windows / Linux / macOS

Hardware Requirements:

- Processor: Intel i3 or higher
- RAM: Minimum 4 GB
- Disk Space: Minimum 2 GB

Theory:

1. Introduction to NoSQL Databases

NoSQL (Not Only SQL) databases are designed to handle large volumes of structured, semi-structured, and unstructured data. Unlike relational databases, NoSQL databases do not enforce fixed schemas and are highly scalable.

Advantages of NoSQL Databases:

- Schema flexibility
- Horizontal scalability
- High performance for big data applications
- Suitable for distributed systems

2. Introduction to MongoDB

MongoDB is a document-oriented NoSQL database that stores data in BSON (Binary JSON) format. Data is stored as documents inside collections, and collections are grouped into databases.

Key Components:

- Database
- Collection
- Document
- Field

3. CRUD Operations in MongoDB

CRUD operations are fundamental database operations:

- Create: Insert documents into a collection
- Read: Retrieve documents from a collection
- Update: Modify existing documents
- Delete: Remove documents from a collection

4. Aggregation in MongoDB

Aggregation is used for data processing and analysis. MongoDB uses an aggregation pipeline, which processes documents through multiple stages.

Common Aggregation Operators:

1. \$match – Filters documents
2. \$group – Groups documents
3. \$sum – Calculates total
4. \$avg – Calculates average

5. Indexing in MongoDB

Indexes improve query performance by allowing MongoDB to quickly locate documents without scanning the entire collection.

Types of Indexes:

1. Single-field index
2. Compound index
3. Unique index

Procedure:

Step 1: Create Database

```
use SalesDB
```

Explanation: Creates and switches to a database named SalesDB.

Step 2: Create Collection

```
db.createCollection("Users")
```

Explanation: Creates a collection named Users to store user data.

Step 3: Insert Documents (Create Operation)

```
db.Users.insertMany([
  { user_id: 1, name: "Amit", age: 25, city: "Pune", sales: 25000 },
  { user_id: 2, name: "Neha", age: 30, city: "Mumbai", sales: 32000 },
  { user_id: 3, name: "Rahul", age: 28, city: "Delhi", sales: 18000 },
  { user_id: 4, name: "Sneha", age: 35, city: "Pune", sales: 40000 },
  { user_id: 5, name: "Karan", age: 22, city: "Ahmedabad", sales: 15000 }
```

```
])
```

Step 4: Read Documents

```
db.Users.find()  
db.Users.find({ city: "Pune" })
```

Explanation: Retrieves all documents and filters users belonging to Pune.

Step 5: Update Documents

```
db.Users.updateOne(  
  { user_id: 3 },  
  { $set: { sales: 20000 } }  
)
```

Explanation: Updates sales value of user with user_id = 3.

Step 6: Delete Documents

```
db.Users.deleteOne({ user_id: 5 })
```

Explanation: Deletes the document where user_id is 5.

Aggregation Queries:

Aggregation 1: Calculate Total Sales

```
db.Users.aggregate([  
  { $group: { _id: null, totalSales: { $sum: "$sales" } } }  
)
```

Aggregation 2: Calculate Average Age

```
db.Users.aggregate([  
  { $group: { _id: null, averageAge: { $avg: "$age" } } }  
)
```

Aggregation 3: City-wise Sales

```
db.Users.aggregate([  
  { $group: { _id: "$city", totalSales: { $sum: "$sales" } } }  
)
```


Indexing:**Step 7: Create Index**

```
db.Users.createIndex({ city: 1 })
```

Step 8: Analyze Query Performance

```
db.Users.find({ city: "Pune" }).explain("executionStats")
```

Observation: Indexed queries execute faster compared to collection scans.

Result:

MongoDB CRUD operations, aggregation queries, and indexing were successfully implemented. Indexing significantly improved query performance.

Conclusion:

MongoDB provides a flexible, scalable NoSQL solution. Aggregation pipelines simplify data analysis, and indexing enhances system performance for large datasets.

Oral Questions:

1. What is NoSQL database?
2. What is MongoDB?
3. Difference between document and relational databases?
4. What is aggregation pipeline?
5. What is indexing and why is it required?
6. What is BSON?
7. How does indexing improve query performance?

Assignment Number - 8**Case Study**

Title: Social Media Data Processing Using NoSQL (MongoDB)

Objective: To process and analyze unstructured social media data using a NoSQL database (MongoDB) and extract meaningful insights such as top trending hashtags using aggregation queries.

Problem Statement: Social media platforms generate massive volumes of unstructured data in the form of posts, comments, likes, hashtags, timestamps, and user metadata. Traditional relational databases struggle to efficiently store and analyze such dynamic data. This case study aims to demonstrate how MongoDB, a NoSQL document-based database, can be used to store, process, and analyze social media data effectively.

Scope of the Case Study:

- Handling unstructured and semi-structured social media data
- Performing analytical queries using MongoDB aggregation pipelines
- Identifying trends such as popular hashtags and active users
- Demonstrating the advantages of NoSQL databases for real-world applications

Software Requirements:

- MongoDB Community Server
- MongoDB Compass / Mongo Shell (mongosh)
- Operating System: Windows / Linux / macOS

Hardware Requirements:

- Processor: Intel i3 or higher
- RAM: Minimum 4 GB
- Storage: Minimum 2 GB free space

Theory:

1. NoSQL and Social Media Data

Social media data is highly unstructured, frequently changing, and generated at high speed. NoSQL databases like MongoDB are ideal for such data because they:

- Do not require a fixed schema
- Support flexible document structures
- Scale horizontally to handle large datasets
- Provide powerful aggregation and indexing features

2. MongoDB Document Model

MongoDB stores data in JSON-like documents (BSON). Each document can have different fields, making it suitable for social media posts where content varies.

Typical Social Media Data Includes:

- Post ID
- User information
- Content text
- Hashtags
- Likes and shares
- Timestamp

Dataset Description:

The dataset contains social media posts with the following attributes:

Field Name	Description
post_id	Unique post identifier
user	Username of the poster
content	Text content of the post
hashtags	Array of hashtags used
likes	Number of likes
shares	Number of shares
post_date	Date of posting

Procedure:

Step 1: Create Database

```
use SocialMediaDB
```

Step 2: Create Collection

```
db.createCollection("Posts")
```

Step 3: Import Dataset

Option 1: Insert Sample Data (For Lab Demonstration)

```
db.Posts.insertMany([
  {
    post_id: 1,
    user: "amit_tech",
    content: "Learning MongoDB is fun! #MongoDB #NoSQL #Database",
    hashtags: ["MongoDB", "NoSQL", "Database"],
    likes: 120,
    shares: 30,
    post_date: ISODate("2024-02-01")
  },
  {
    post_id: 2,
    user: "neha_codes",
    content: "AI is transforming the future #AI #MachineLearning",
    hashtags: ["AI", "MachineLearning"],
    likes: 250,
    shares: 60,
    post_date: ISODate("2024-02-02")
  },
  {
    post_id: 3,
    user: "rahul_dev",
    content: "Big data analytics using NoSQL #BigData #NoSQL",
    hashtags: ["BigData", "NoSQL"],
    likes: 180,
```

```

    shares: 45,
    post_date: ISODate("2024-02-03")
  },
  {
    post_id: 4,
    user: "sneha_ai",
    content: "Deep learning trends in 2024 #AI #DeepLearning",
    hashtags: ["AI", "DeepLearning"],
    likes: 300,
    shares: 90,
    post_date: ISODate("2024-02-04")
  }
])

```

Step 4: Read All Posts

```
db.Posts.find().pretty()
```

Data Analysis Queries:

Query 1: Find Posts with More Than 200 Likes

```
db.Posts.find({ likes: { $gt: 200 } })
```

Query 2: Find Posts Containing a Specific Hashtag

```
db.Posts.find({ hashtags: "AI" })
```

Query 3: Find Top Trending Hashtags

```

db.Posts.aggregate([
  { $unwind: "$hashtags" },
  { $group: { _id: "$hashtags", count: { $sum: 1 } } },
  { $sort: { count: -1 } },
  { $limit: 5 }
])

```

```
])
```

Explanation:

\$unwind breaks hashtag arrays into individual documents

\$group counts hashtag occurrences

\$sort ranks hashtags by popularity

\$limit displays top trending hashtags

Query 4: User Engagement Analysis

```
db.Posts.aggregate([
  {
    $project: {
      user: 1,
      total_engagement: { $add: ["$likes", "$shares"] }
    }
  }
])
```

Indexing for Performance Optimization:**Create Index on Hashtags**

```
db.Posts.createIndex({ hashtags: 1 })
```

Analyze Query Performance

```
db.Posts.find({ hashtags: "AI" }).explain("executionStats")
```

Observation: Indexed searches execute faster than collection scans, especially with large datasets.

Result:

The social media dataset was successfully imported into MongoDB. Aggregation queries efficiently identified trending hashtags and user engagement patterns.

Conclusion:

MongoDB effectively handles unstructured social media data. Aggregation pipelines enable powerful analytics, and indexing significantly improves performance, making NoSQL databases ideal for real-time social media applications.

Real-World Applications:

1. Trend analysis on social media platforms
2. Sentiment analysis systems
3. Recommendation engines
4. Marketing analytics

Oral Questions:

1. Why is NoSQL suitable for social media data?
2. What is \$unwind in MongoDB aggregation?
3. How are trending hashtags identified?
4. What is the advantage of indexing in MongoDB?
5. Difference between SQL and NoSQL databases?
6. What is BSON?

Assignment Number - 9

Title: MySQL Database Backup and Recovery

AIM: To understand and practice the process of taking backup and restoring a MySQL database to ensure data security, integrity, and disaster recovery.

Problem Statement: Databases are critical assets for organizations and are vulnerable to failures such as system crashes, accidental deletions, hardware failures, and cyber-attacks. This experiment focuses on implementing backup and recovery mechanisms in MySQL to prevent data loss and ensure business continuity.

Objective: To perform database backup and recovery operations in MySQL using command-line tools and understand their importance in real-world database management.

Task:

- Create a MySQL database with sample data

- Take a full database backup

- Simulate data loss

- Restore the database from backup

- Verify data recovery

Objectives of the Experiment:

- To understand the importance of database backup

- To learn different MySQL backup techniques

- To perform database restoration

- To ensure data integrity after recovery

- To gain hands-on experience with MySQL administration commands

Outcome:

After completing this experiment, students will be able to:

- Take full backups of MySQL databases
- Restore databases using backup files
- Protect data against accidental loss
- Understand disaster recovery concepts
- Manage databases efficiently in real-world scenarios

Software Requirements:

- Database: MySQL Server 8.0 or above
- Client Tool: MySQL Command Line / MySQL Workbench
- Operating System: Windows / Linux / macOS

Hardware Requirements (Minimal):

- Processor: Intel i3 or equivalent
- RAM: 4 GB or more
- Storage: 2 GB free disk space
- Display: 1024 × 768 resolution or higher

Theory:**1. Introduction to Database Backup**

A database backup is a copy of data that can be used to restore the original database in case of data loss. Backup strategies are essential for:

- Disaster recovery
- Data migration
- System failures
- Accidental deletions

2. Types of MySQL Backup**Logical Backup:**

Uses SQL statements to recreate data

Example: mysqldump

Physical Backup:

Copies actual data files

Faster for large databases

This experiment focuses on logical backup using mysqldump.

3. MySQL Backup Tool – mysqldump

mysqldump is a MySQL utility used to create logical backups of databases in the form of SQL scripts that can later be restored.

Procedure:

Step 1: Create Database

```
CREATE DATABASE CompanyDB;  
USE CompanyDB;
```

Step 2: Create Table

```
CREATE TABLE Employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(100),  
    department VARCHAR(50),  
    salary DECIMAL(10,2)  
);
```

Step 3: Insert Sample Data

```
INSERT INTO Employees VALUES  
(1, 'Amit Sharma', 'IT', 50000),  
(2, 'Neha Patil', 'HR', 45000),  
(3, 'Rahul Mehta', 'Finance', 55000),  
(4, 'Sneha Kulkarni', 'IT', 60000);
```

Step 4: Verify Data

```
SELECT * FROM Employees;
```

Backup Process:**Step 5: Take Database Backup****Command (Executed in Command Prompt / Terminal):**

```
mysqldump -u root -p CompanyDB > CompanyDB_backup.sql
```

Explanation:

1. `mysqldump` Backup utility
2. `-u root` MySQL username
3. `-p` Prompt for password
4. `CompanyDB` Database name
5. `CompanyDB_backup.sql` Backup file

Simulating Data Loss:**Step 6: Drop the Database**

```
DROP DATABASE CompanyDB;
```

Step 7: Verify Database Deletion

```
SHOW DATABASES;
```

Recovery Process:**Step 8: Restore Database from Backup**

```
mysql -u root -p CompanyDB < CompanyDB_backup.sql
```

Step 9: Verify Restoration

```
USE CompanyDB;  
SELECT * FROM Employees;
```

Result:

The MySQL database backup was successfully created using mysqldump. After simulating data loss, the database was restored correctly, and all records were recovered without any data loss.

Conclusion:

Backup and recovery are essential components of database administration.

mysqldump provides a reliable logical backup mechanism in MySQL.

Regular backups ensure data safety and business continuity.

Database recovery helps organizations handle disasters efficiently.

Applications:

Enterprise database management

Disaster recovery systems

Cloud database migration

Data security and compliance

Oral Questions:

1. What is database backup?
2. Why is backup important in DBMS?
3. What is mysqldump?
4. Difference between logical and physical backup?
5. What happens if backup is not taken regularly?
6. How do you restore a MySQL database?
7. What is disaster recovery?